Developing a Large Language Model in Meitei Mayek for the Manipuri Language

A B.Tech project report submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

by

Nelson Singh Shagolsem (22103067)
Ronaldo Nandeibam (22103071)

Under the Supervision of

Dr. Dennis Singh Moirangthem



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY MANIPUR

Imphal - 795004, India

May, 2026

Department of Computer Science and Engineering
NATIONAL INSTITUTE OF TECHNOLOGY MANIPUR
Langol, Imphal. Ph. (0385) 2445812 Email hodcse@nitmanipur.ac.in
(An Autonomous Institute under Ministry of Education, Govt. of India)

No.NITMN.3/(89-Acad)/CSE/B.Tech/Project/2026/19 Date: 18/05/2026

Certificate

This is to certify that the report entitled **"Developing a Large Language Model in Meitei Mayek for the Manipuri Language"**, submitted by **Nelson Singh Shagolsem** (22103067) and **Ronaldo Nandeibam** (22103071) to the National Institute of Technology Manipur, for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, is a record of the original, bonafide research work carried out by them under our supervision and guidance. The report has reached the standards fulfilling the requirements of the regulations related to the award of the degree.

The results contained in this report have not been submitted in part or in full to any other University or Institute for the award of any degree or diploma to the best of our knowledge.

18/5/26

Dr. Dennis Singh Moirangthem
Supervisor,
Designation (Assistant Professor),
Department of CSE, NIT Manipur.

18/5/2026

Dr. Yambem Jina Chanu
Head of Department,
Associate Professor,
Department of CSE, NIT Manipur.

Head of Department
Department of CSE

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Nelson Singh Shagolsem
.....

Nelson Singh Shagolsem
Roll No.: 22103067

RonaldoN.
.....

Ronaldo Nandeibam
Roll No.: 22103071

.....

Date: 18/5/22
Place: NIT Manipur

Acknowledgements

We would like to express our sincere gratitude to all those who supported and guided us throughout the course of this project.

We would like to express our special thanks of gratitude to our project supervisor, **Dr. Dennis Singh Moirangthem** , Assistant Professor, Department of Computer Science and Engineering, National Institute of Technology Manipur, who gave us the opportunity to work on this project and for his continuous support, guidance, and encouragement throughout. We are especially grateful for his generosity in providing access to the NVIDIA RTX PRO 5000 Blackwell GPU, without which the primary training experiment of this dissertation would not have been possible.

We are also thankful to **Dr. Yambem Jina Chanu**, Head of the Department of Computer Science and Engineering, NIT Manipur, for providing us everything we required to complete our project and for fostering a research environment that made this work possible.

We extend our thanks to the open-source community whose tools and resources formed the backbone of this project including the developers of the Hugging Face ecosystem, the SentencePiece library and the PyTorch framework.We would also like to thank the authors of "A Benchmark Dataset for Printed Meitei/Meetei Script Character Recognition" and Dayananda Thokchom for creating the english-TO-meitei-mayek dataset publicly available. Our research would not have been possible without these.

We want to thank e-pao.net for creating and maintaining a publicly available corpus of Manipuri (Meitei Mayek) news articles ranging over seventeen years. This was used as part of the training corpus.

Nelson Singh Shagolsem
(Roll No.: 22103067)

Ronaldo Nandeibam
(Roll No.: 22103071)

Date: 18/5/26

Place: NIT Manipur

Abstract

The rise of Large Language Models in recent years has further exacerbated the severe technology gap faced by low-resource languages. Manipuri (officially known as Meiteilon), one of the 22 scheduled languages of India written in the native script of Meitei Mayek, has barely been touched by NLP applications. Even then, nearly all of Manipuri NLP has been limited to translation tasks and models trained on the Bengali script instead of Meitei Mayek. There has been no attempt whatsoever at creating a generative model for Manipuri text written in the native script of Meitei Mayek.

Until now. This project aims to create said model. There were three phases of experimentation. Experiment 1 took Dayananda Thokchom's Hugging Face dataset "english-TO-meitei-mayek" consisting of 401,757 cleaned monolingual sentences and fine-tuned BLOOMZ-1B1 with LoRA on a pilot set of 1,000 sentences. Output text was nearly perfect Meitei Mayek script but barely any semblance of semantics. Experiment 2 trained a custom Mini-GPT from scratch with 27.1 million parameters on my RTX 4060 laptop. Semantic coherence improved along with its ability to learn Meitei Mayek script structure, but was clearly overfitting. Experiment 3, the main experiment, created a corpus from scratch by scraping news articles from the website e-pao.net (archives dating as far back as 2008) and OCR text from the Mendeley Printed Meitei Meetei Document Images Benchmark Dataset which contains 11 books and newspaper pages. The e-pao.net data required a full legacy font conversion pipeline — the Ratha font encodes Meitei Mayek glyphs as ASCII characters, requiring a manually reconstructed character mapping to recover proper Unicode text. The merged and cleaned corpus yielded 163,834 lines and approximately 25.3 million tokens.

A SentencePiece BPE tokenizer with 2,000 tokens was selected through coverage analysis across 25 vocabulary sizes, achieving 92.7% token frequency coverage with zero unknown tokens. A direct experiment confirmed that fully replacing Mistral 7B's original vocabulary with this tokenizer outperformed vocabulary extension, with a perplexity of 153 versus 356. Mistral 7B was then fine-tuned using LoRA with rank 32 and alpha 64 for 10,000 steps on an NVIDIA RTX PRO 5000 Blackwell

GPU. Final train loss reached 5.58 against a validation loss of 5.61 — a gap of 0.03 indicating no overfitting. Against 500 unseen test samples, the untrained baseline scored a perplexity of 624,031 while the trained model scored 210, a reduction of 99.97%. Generated outputs are entirely in Meitei Mayek script with no English contamination.

This work contributes a functional generative model, a cleaned corpus, a validated tokenizer, and an end-to-end replicable pipeline for Meitei Mayek — a foundation others can build on for this underserved language.

Contents

Certificate

Declaration

Acknowledgements

Abstract

Contents

List of Figures

List of Tables

1 Introduction **1**
- 1.1 Problem Statement 1
- 1.2 Need of the Study 2
- 1.3 Study Objective 3
- 1.4 Dissertation Organization 4
- 1.5 Summary 4

2 Literature Review **6**
- 2.1 Literature Review 6
  - 2.1.1 The Evolution of Large Language Models 6
  - 2.1.2 The Low-Resource Language Challenge 7
  - 2.1.3 The State of Manipuri Natural Language Processing 7
- 2.2 Research Gaps 7
- 2.3 Summary 8

3 Study Methodology **9**
- 3.1 Introduction 9
- 3.2 Models and Algorithm 10
  - 3.2.1 Transformer Model Architecture 10
  - 3.2.2 Choosing the Base Model 11

3.2.3 Tokenizer Design . . . 12
3.2.4 Training Setup . . . 13
3.3 Evaluation Framework . . . 14
3.4 Summary . . . 15

4 Data Collection, Corpus Construction and Tokenization **17**
4.1 Data Source Selection . . . 17
4.1.1 HuggingFace English-to-Meitei-Mayek Dataset . . . 18
4.1.2 E-Pao News Archive . . . 18
4.1.3 A Benchmark Dataset for Printed Meitei/Meetei Script Character Recognition . . . 19
4.2 Data Collection and Extraction . . . 19
4.2.1 The Legacy Font Problem . . . 19
4.2.2 Font Conversion Pipeline . . . 20
4.3 Corpus Cleaning and Merging . . . 20
4.4 Tokenizer Design and Selection . . . 21
4.4.1 Background . . . 21
4.4.2 Training Tokenizers Across 25 Vocabulary Sizes . . . 22
4.4.3 Coverage and Compression Analysis . . . 22
4.4.4 Vocabulary Size Selection . . . 25
4.4.5 Vocabulary Integration Strategy . . . 25
4.5 Summary . . . 25

5 Experiments, Analysis and Results **27**
5.1 Experimental Setup . . . 27
5.2 Experiment 1 — BLOOMZ-1B1 Fine-Tuning . . . 27
5.2.1 Results . . . 28
5.2.2 Observations and Limitations . . . 29
5.3 Experiment 2 — Custom Mini-GPT from Scratch . . . 30
5.3.1 Results . . . 31
5.3.2 Observations and Limitations . . . 32
5.4 Experiment 3 — Mistral 7B Main Training . . . 32
5.4.1 Model Selection . . . 32
5.4.2 Tokenizer Strategy: Replacement vs Extension . . . 33
5.4.2.1 Results . . . 34
5.4.3 Mistral 7B Main Training Setup . . . 35
5.4.4 Training Results . . . 36
5.5 Final Evaluation and Baseline Comparison . . . 40
5.5.1 Setup . . . 40
5.5.2 Results . . . 40
5.5.3 Inference and Generated Samples . . . 41
5.6 Discussion . . . 42

5.7 Summary 43

6 Conclusions and Practical Implications **45**
6.1 Study Findings 45
6.2 Research Contributions 46
6.3 Practical Implications 47
6.4 Study Limitations and Scope for Future Work 48

Bibliography **50**

List of Tables

4.1 Corpus Split Statistics 21
4.2 Tokenizer Coverage Summary (Selected Vocabulary Sizes) 24

5.1 Experiment 1 Training Configuration 28
5.2 Mini-GPT Architecture Configuration 30
5.3 Tokenizer Strategy Comparison Results at 1,000 Steps 34
5.4 Mistral 7B Training Configuration 36
5.5 Mistral 7B Training Metrics at Selected Steps 39
5.6 Final Evaluation: Untrained Baseline vs Trained Model 40

Chapter 1

Introduction

The Large Language Models which rely on the Transformer architecture [12] have transformed how people communicate with computers. It is honestly a great change in English and other dominant languages. But the progress is not the same everywhere, it does left a bigger gap between languages loaded with digital data and those pretty much left out. Languages which have rich history and culture are still severely under represented in the modern AI era.

This dissertation explores the aforementioned gap through Manipuri, one of the 22 scheduled languages of India and the official language of Manipur, making it significant both culturally and administratively. However, digitally it remains almost nonexistent.The focus here is to build a solid, high-performing Large Language Model for Manipuri in the Meitei Mayek script, with a hope to boost its presence in AI.

1.1 Problem Statement

There is no generative, high-performance Large Language Model for Manipuri in the Meitei Mayek script exist. And there are three big reasons why:

- **Massive Data Gap:** We need a very huge amount of clean and accessible text to train any language model. The challenges of data scarcity faced by

low-resource languages in building a language model are well-documented in the literature [8]. For Manipuri (especially in Meitei Mayek), the digital text does not exist at all. So, it will be difficult to train and build a decent model from scratch.

- **Script Disconnect:** Most of the NLP work uses Bengali script for Manipuri, even in the advanced stuff also, as we have seen in the work of Singh and Bandyopadhyay [10]. That is convenient in a way but completely ignore the real and culturally important script. Though it is officially recognized as a scheduled language but it is barely supported digitally, which is a huge mismatch.

- **No Generative Models at All:**The Manipuri NLP is remain stuck at translation stage and there are no generative models trained on Manipuri that can actually handle its nuances. So, there exist not a single chatbots, no content creators and even no local AI-powered educational tools in the native script(Metei Mayek).

1.2 Need of the Study

This is not just about some academic exercise. It is also about protecting our culture, improving education and also giving Manipuri speakers a real chance. Here is why this work we have been doing is important:

- **Preserving Language and Culture:** The language tech always helps dominant languages to get even bigger than it was [2]. A Manipuri LLM could actually help digitize the old texts, generate new content and power platforms which use the Meitei Mayek directly.

- **Transforming Education:** A Manipuri LLM what we mean is to have better tools in AI tutors, grammar checkers, interactive modules in the actual language [2]. With it's help students will learn better and build literacy with tools that make more sense for them. [8].

- **Tech Equity:** Having serious AI tools in Manipuri let the local developers, businesses and government teams to build services in chatbots, public info systems that are actually usable for everyone. That's what real tech inclusion looks like in this modern AI era.
- **Moving NLP Forward:** This project we are moving forward is not content to only Manipuri-Metei Mayek. The dataset, tokenizer and training pipeline we are developing can also help other low-resource languages join the tech world.

1.3 Study Objective

The big goal is to build, train and test a generative Large Language Model that writes fluent Manipuri in Meitei Mayek script.

Here is the breakdown:

1. Start by building a quality monolingual Meitei Mayek text corpus from translation datasets, different web archives and other digitized documents available.
2. Design and train a custom tokenizer that can fit the Meitei Mayek's script and structure and choose the best vocabulary size based on the actual coverage.
3. Experiment across different model setups which involve fine-tuning of massive multilingual models and also training a Mini-GPT from scratch to find what works best under the resource constraints and chose the most efficient one.
4. Fine-tune a top language model with LoRA for efficient adaptation of the language and also need to aim for strong performance without overfitting.
5. Test the trained model and evaluate against different evaluation matrices which involve checking baseline using loss, perplexity, and check if the generated text is correct and coherent in the script.
6. Document everything on the journey whether in corpus building, tokenizer set up, training and evaluation so that someone else can pick it up and run with

it or use it as an information to build language model for other low resource language.

1.4 Dissertation Organization

There are six chapters and each has a clear role:

Chapter 1 sets up the motivations to work on this project, the context of it, the big problem and challenges faced in building it and the main objectives to achieve it.

Chapter 2 reviews what's already out there which include the LLMs, low-resource NLP, Manipuri language tech and highlights the missing pieces that this project aims to fill in.

Chapter 3 gets into the methodology and technical stuff which states the Transformer models, LoRA fine-tuning, SentencePiece tokenization and all the design decisions that we proceed with.

Chapter 4 details the data pipeline which states where the data came from, how it was collected by us, the font conversions, cleaning, merging, and tokenizer training and all sources and choices available.

Chapter 5 covers the key experiments we involve with which include BLOOMZ fine-tuning, Mini-GPT building, and Mistral 7B. It shares the results, training flows, evaluations of it and dives into the generated outputs.

Chapter 6 wraps everything up, it explains what has been achieved so far, what it would mean for the Manipuri-Metei Mayek and NLP and lays out the future directions with the base foundation we have built.

1.5 Summary

Manipuri already has an official status and a distinctive script, but there is zero generative AI support. Everything related to AI remain stuck at translation and

uses Bengali script which is not the native script for the Manipuri language. This leave us a clear gap in both research and technology for everything combining low-resource NLP and indigenous script preservation.

The objectives here is to guide the whole project route that includes building a usable corpus, creating a custom tokenizer, run all solid experiments and deliver a generative model that's documented and easy for others to work with. Onward, Chapter 2 looks at the research that shaped this work and pinpoints the gaps that this study tries to close.

2.1.2 The Low-Resource Language Challenge

This pre-training approach works best for high-resource languages that has a lots of data. Manipuri. Meanwhile it become a struggle for low-resourse languages.

To fix this, researchers built huge multilingual models like XLM-RoBERTa, as described by Conneau et al. [3]. These cover lots of languages at once. Still, they don't really capture the unique features of Manipuri unless you fine-tune them. Even then, Manipuri barely gets any representation because there's just not much data in the mix.

Fine-tuning these big models used to be expensive and slow. But Parameter-Efficient Fine-Tuning (PEFT) changed things up. LoRA, introduced by Hu et al. [6], became the hero. It keeps most of the model frozen and just tweaks a few trainable pieces. That reduction in costs make it possible to specialize big models even when there's not much data.

2.1.3 The State of Manipuri Natural Language Processing

Manipuri NLP is starting to move, but the focus has been still narrow. Researchers like Thoudam Doren Singh and Sivaji Bandyopadhyay [10] mostly worked on machine translation.

There is two big problems stand out. First, almost all the studies are about translation, not about generating new Manipuri text. Second, they used Bengali script instead of native Meitei Mayek script.

2.2 Research Gaps

After looking into the literature we found out three major gaps.

1. **The Generative Gap:** Nobody has ever tried building a generative model for Manipuri. Even when we try looking up to the work done related in this field

we found out it is all just translation and there is no fresh Manipuri content similar to what we are initiating.

2. **The Scriptural Gap:** The existing Manipuri NLP tools work with Bengali script not with the Meitei Meyek script even though Meitei Mayek is the official which we need to take initiative about it to take over the Bengali by the Metei Meyek for the Manipuri NLP tools. There is not a single high performing language model for Manipuri in Meitei Mayek.

3. **The Specialization Gap:** Multilingual models technically somtime include Manipuri but they include it barely. No one has ever fine-tuned a foundational model which was done with a carefully picked Manipuri dataset and customized with custom tokenizer. That is how we get high-quality results for low-resource languages.

2.3 Summary

This chapter go through how the main technologies had evolved from Transformers to the established routine of pre-training and fine-tuning. We have also placed the Manipuri NLP within the broader challenge of low-resource languages and checked out what research has happened so far.

The tools already exist, but nobody has ever built a generative model for Manipuri in Meitei Mayek. Finding out this gaps we perform generative modeling, supporting the right script and model specialization which shows exactly why the research in the next chapters matters.

Chapter 3

Study Methodology

3.1 Introduction

The overall goal was to build a language model that can read and generate text in the Meitei Mayek script — something that has never been done before. Since there was no existing template or prior work to follow for this specific language and script, the methodology was designed to be exploratory and iterative, meaning each experiment informed the next one. The approach explored two broad strategies. The first is transfer learning — start ing with a model that has already been trained on large amounts of text in other languages and fine-tuning it on Meitei Mayek data. The second strategy, explored in the second experiment, was training a model completely from scratch using only Meitei Mayek data, with no pretrained weights at all. This gave a direct comparison point between the two approaches.

Figure 3.1 shows the architecture of the Transformer model, which is the foundation that all modern language models — including both the pretrained models fine-tuned and the custom model built from scratch — are built on.

The whole project unfolded in three phases: first, picking the best base model; next, adapting it for Meitei Mayek with a method called LoRA; then, finally, testing to see how well the model worked.

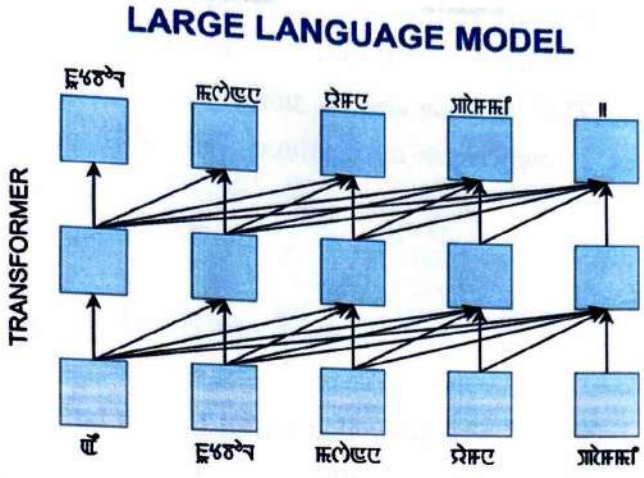


FIGURE 3.1: Architecture of the Transformer Model [12]

3.2 Models and Algorithm

3.2.1 Transformer Model Architecture

Each of the three modeling efforts in this project experimented with variations of the Transformer model architecture [12], using it either as a pretrained base model or as the starting point for a bespoke architecture implementation.

The Transformer feeds a sequence of tokens representing the input text through a series of embedded layers. Each token is embedded into a dense vector embedding which is passed through N identical layers. Each layer consists of two major sub-layers: a multi-head self attention block and a position-wise feed-forward neural network.

Self attention allows each token in the input sequence to directly attend to every other token in the sequence. The "multi-head" qualifier indicates that these sub-layers are duplicated and run in parallel with each attention "head" learning to independently attend to different features in the relationship between tokens.

By dispensing with the recursive processing of prior RNN-based architectures, transformers allowed training to proceed massively in parallel, dramatically speeding up training and allowing model sizes to grow to consume massively larger datasets.

Mistral 7B [7], emerged as the strongest candidate. Despite having 7 billion param eters nearly half of LLaMA 2 13B, it outperforms that model across all evaluated benchmarks, and further outperforms the much larger LLaMA 1 34B on reasoning, mathematics, and code generation tasks. This is achieved through two architec tural improvements. Sliding Window Attention allows the model to handle longer sequences more efficiently by limiting direct attention to a local window while still propagating information through layer stacking. Grouped Query Attention reduces the memory required for the key-value cache during generation by sharing key and value projection heads across groups of query heads. Both features made Mistral 7B more memory-efficient than its parameter count alone would suggest, which was directly relevant given the hardware constraints of this project.

That's thanks to features like Sliding Window Attention (great for longer texts) and Grouped Query Attention (saves memory during text generation). When you're working with limited GPU memory, that stuff matters.

Prior work [13] further confirms that Mistral 7B can be successfully adapted to local language corpora through continued pretraining, providing evidence that the architecture handles low-resource language specialization effectively even when the target language has limited representation in the original pretraining data. Taking all of these factors together,Mistral 7B was selected as the base model for the primary experiment

3.2.3 Tokenizer Design

Before any model can be trained, the text has to be broken into tokens — the ba sic units the model works with. For Meitei Mayek, built-in tokenizers of pretrained models are essentially useless. They were not trained on Meitei Mayek text and frag ment the script inefficiently, sometimes breaking individual characters into multiple pieces and producing sequences far longer than necessary.

The solution across all experiments was to train a custom tokenizer from scratch using the SentencePiece library. SentencePiece works by starting with individual characters and repeatedly merging the most common pairs of adjacent symbols until it reaches the desired vocabulary size. The result is a vocabulary where common

Meitei Mayek words and syllables appear as single tokens, which helps the model learn the language more efficiently

The vocabulary size was handled differently across the experiments. In the first experiment with BLOOMZ, a vocabulary of 32,000 tokens was chosen based on what seemed standard for a multilingual model of that scale. In the second experiment with the custom Mini-GPT, this was reduced to 8,000 tokens. Neither of these choices was backed by empirical analysis of the specific corpus.

In the third and final experiment, we conducted dedicated vocabulary size investigation. Twenty-five separate tokenizers were trained with vocabulary sizes ranging from 900 to 30,000 tokens and evaluated systematically. The analysis, described in full detail in Chapter 4, showed that a vocabulary of 2,000 tokens provided the best balance between coverage quality and compression efficiency — 92.7a single unknown token appeared across any of the three data splits. This was the vocabulary used for the Mistral 7B training.

3.2.4 Training Setup

The training configuration varied across the three experiments as the project pro gressed.

In the first experiment, BLOOMZ-1B1 was fine-tuned on a pilot subset of 1,000 sentences for approximately 24 epochs using the PyTorch framework on Google Colab. The optimizer was AdamW and LoRA was applied to the four attention projection layers with rank 8 and alpha 32. Gradient checkpointing was enabled to reduce memory usage.

The second experiment trained the custom Mini-GPT entirely from scratch on the full 401,757-sentence corpus. This used the AdamW optimizer with a learning rate of 3×104, a batch size of 64, and ran for 3 epochs on an NVIDIA RTX 4060 Laptop GPU with 8GB of VRAM. Total training time was approximately 12.5 hours.

The third and primary experiment used a more carefully designed configuration. Mistral 7B was fine-tuned using AdamW with a peak learning rate of 2×105, following a cosine decay schedule with a linear warmup over the first 400 steps.

Training used a per-device batch size of 2 sequences with gradient accumulation over 16 steps, giving an effective batch size of 32. The maximum sequence length was set to 1,024 tokens, and gradient clipping was applied with a maximum norm of 0.5. Training ran for 10,000 steps on an NVIDIA RTX PRO 5000 Blackwell GPU with 48GB of VRAM provided by the project supervisor.

3.3 Evaluation Framework

Evaluating a language model ,especially one for a language with no prior bench marks, requires being clear about what the numbers do and do not tell you.

Evaluating a brand new language model for a script nobody has modeled before means the usual metrics only tell part of the story.

The main quantitative metric that we used was perplexity. Perplexity measures how well the model's predicted probability distribution over the vocabulary matches the actual next token at each position. It is defined as:

$$\text{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid w_1, \ldots, w_{i-1})\right) \tag{3.1}$$

A lower perplexity indicates better predictive confidence. For a vocabulary of 2,000 tokens, a completely untrained model would produce a very high perplexity because every token is equally unexpected.

Besides perplexity, validation loss was monitored during the training. Validation loss is the cross-entropy loss calculated on the held-out validation set without gradient computation. If the training loss keeps falling and the validation loss starts rising, the model is overfitting i.e memorizing the training data rather than generalizing the language patterns. The gap between training loss and validation loss is therefore as important as their individual value.

The final evaluation compared the trained model directly with the untrained Mistral 7B baseline with replaced 2k vocabulary. This comparison on 500 unseen test sentences gives a clean measure of how much the fine-tuning actually contributed.

Also generated text samples were examined qualitatively. We checked whether the output was in the correct script, whether it has any other languages contamination like English. We also examined whether the text showed signs of meaningful linguistic structure.

3.4 Summary

This chapter described the methodology that we used to develop a generative language model for Meitei Mayek. Since no previous model existed for this script, our work consists of experimental and iterative process.

We conducted three experiments. In first experiment we fine-tuned BLOOMZ-1B1 on a small dataset to test feasibility. In second experiment we trained a custom Mini-GPT style Transformer from scratch on Meitei Mayek data. And in the final and main experiment we used Mistral 7B because of its strong performance and efficient architecture.

Custom tokenizers were trained using SentencePiece because existing tokenizers handled Meitei Mayek poorly as they have not seen any Meitei Mayek before . After we tested vocabulary sizes from 900 to 30,000, a vocabulary of 2,000 tokens was selected as the best balance between efficiency and coverage. Training setups improved across experiments as we continued. We trained final Mistral 7B model using LoRA, AdamW optimization, cosine learning rate decay, gradient accumulation, and training on a 48GB NVIDIA RTX PRO 5000 GPU. In evaluation we used perplexity, validation loss, and qualitative analysis of generated text to measure the quality of Meitei Mayek learning by the model. We were also able to avoid overfitting.

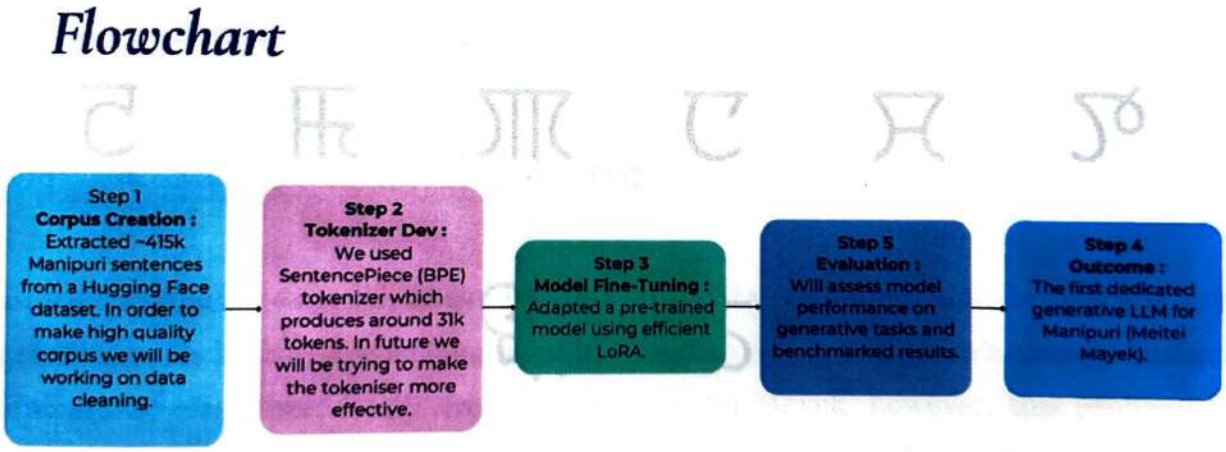


FIGURE 3.2: Overview of the Project Methodology Pipeline

Chapter 4

Data Collection, Corpus Construction and Tokenization

4.1 Data Source Selection

To train a language model, one must start with data. Obtaining data is typically not a challenge for high-resource languages like English — after all there's already billions of words of clean text freely available. For Meitei Mayek, however, this problem is fundamentally different. Digitized text written in the native Meitei Mayek script is limited, fragmented across sources of various formats, and in some cases encoded in legacy font encodings that need conversion just to be usable. Acquiring enough clean usable text was a major hurdle of this project.

The approach to the data changed over the course of experimentation.For the first two experiments, a single publicly available parallel dataset was used — the english-TO-meitei-mayek translation corpus from Hugging Face. While this dataset provided a reasonable starting point given the scarcity of Meitei Mayek text, it came with an inherent limitation: being a translation dataset, it consisted of sentence pairs aligned to English source sentences. This meant the Meitei Mayek sentences, when extracted in isolation, were not naturally flowing continuous text. Each line was an independent translation unit rather than part of a coherent narrative or article, so there was little to no contextual relationship between consecutive lines in

the corpus. For a generative language model that needs to learn how ideas connect and flow across sentences, this is a meaningful shortcoming.

This limitation was one of the motivating factors behind switching to entirely new data sources for the primary experiment. The e-pao.net news archive and the Mendeley printed document dataset both provide naturally flowing, contextually coherent text, news articles with continuous paragraphs and printed books with connected prose.

4.1.1 HuggingFace English-to-Meitei-Mayek Dataset

The first data source used in this project was the "english-TO-meitei-mayek" dataset published on Hugging Face by Dayananda Thokchom[11]. This is a parallel corpus containing aligned English and Meitei Mayek sentence pairs, designed to support low-resource NLP tasks including translation and text generation. The dataset contained approximately 415,212 sentence pairs in its full form.

Since the objective of this project is monolingual text generation rather than translation, only the Meitei Mayek column was extracted. The English side was discarded entirely. After removing empty rows, duplicate sentences, entries containing English contamination, and lines written in the Bengali script rather than Meitei Mayek, the cleaned monolingual corpus contained 401,757 sentences. This corpus was used for both the BLOOMZ fine-tuning experiment and the custom Mini-GPT training.

4.1.2 E-Pao News Archive

We built another corpus from scratch for our initial Mistral 7B experiment. This corpus came from two separate websites. The first was e-pao.net [4], a relatively small online Manipuri news outlet that prints its articles in Meitei Mayek script. However, the website hosts a public archive of news articles which have been published since December 2008 where the articles are grouped by date published.

We wrote a custom scraper in Python with the `requests` package that would loop through every day between December 7, 2008 and today, programmatically building

the archive URL for that day, downloading the resulting page, and using regular expression pattern matches on custom HTML tags to harvest article headings and body text. Heads and bodies of articles were cleaned independently (removing any html tags, JavaScript code blocks, Cloudflare page artifacts, and source attribution). In total, we pulled over 17 years worth of content from December, 2008 to 2026.

With our custom scraper pipeline, we extracted over 46,989 news articles from thousands of archive pages of news content, resulting in one of the largest Meitei Mayek corpora.

4.1.3 A Benchmark Dataset for Printed Meitei/Meetei Script Character Recognition

The second novel source was "A Benchmark Dataset for Printed Meitei/Meetei Script Character Recognition" [9]. This was a research dataset of scanned images of documents printed in Meitei Mayek, together with their corresponding transcribed text files.The dataset covers eleven books — BOOK_1, BOOK_2, BOOK_3, BOOK_4, BOOK_7, BOOK_8, BOOK_10, BOOK_11, BOOK_15, BOOK_16, and BOOK_20 — as well as a collection of newspaper pages.

We wrote a Python script and traverse the dataset directories and identify the transcription text associated with each scanned document. And then we extract the Unicode Meitei Mayek text content.

4.2 Data Collection and Extraction

4.2.1 The Legacy Font Problem

Upon scraping and analysing the data for e-pao.net we noticed an issue. The scraped text was displaying Latin characters instead of Meitei Mayek. The text wasn't corrupt, it was encoded with an ancient font known as ratha.ttf.

This font file assigns normal ASCII characters to the shape of Meitei Mayek glyphs that we see essentially when you press "a" on your keyboard, The Ratha font shows

you what resembles a Meitei Mayek letter. The underlying file, however, stores the ASCII character "a", not the Unicode Meitei Mayek character. When this text is opened without the Ratha font installed, it appears as meaningless Latin letters. This meant the scraped e-pao.net text could not be used directly for training. A conversion pipeline had to be built to transform the Ratha-encoded ASCII text into proper Unicode Meitei Mayek.

4.2.2 Font Conversion Pipeline

The conversion was carried out in two steps. The first step handled structural markers that the scraper had embedded in the corpus — labels like "HEADING", "BODY", and "Article" which were written in plain English and would interfere with the character-level mapping if left unchanged. These were first replaced with their Ratha-encoded equivalents using a replacement JSON file, so that they would survive the conversion process correctly.

The second step applied the character mapping itself. The Ratha font file was analyzed using FontForge, an open-source font inspection and editing tool. FontForge made it possible to inspect how individual Latin characters were mapped internally to Meitei Mayek glyphs inside the legacy font. By comparing the displayed glyphs with their intended Unicode Meitei Mayek equivalents, a manual mapping table was constructed. This mapping associated each legacy encoded character with its correct Unicode representation. This mapping was recorded in a JSON file covering all lowercase letters, uppercase letters, digits, and punctuation characters. A Python conversion script then read the raw corpus character by character, looked up each character in the mapping table, and wrote the corresponding Unicode character to the output file. Characters not found in the mapping were passed through unchanged and logged as unknowns for review.

4.3 Corpus Cleaning and Merging

After the e-pao.net data had been converted to Unicode and the Mendeley data had been extracted, the two corpora were merged into a single file. The merged corpus

was then passed through a cleaning pipeline that removed structural markers, normalized spacing, removed blank lines, and filtered out any remaining lines that did not contain Meitei Mayek Unicode characters. Lines containing only punctuation, numbers, or whitespace were also removed at this stage.

The final merged and cleaned corpus contained 163,834 lines of pure Meitei Mayek text. This corpus combined three distinct text types: web-scraped news articles covering a wide range of current affairs topics, printed book text covering more formal and literary registers, and newspaper content. This diversity was intentional — a corpus drawn from a single source and register would likely produce a model that generates text in only one style, which limits its usefulness.

The corpus was then split into training, validation, and test sets using a fixed random seed of 42 to ensure reproducibility. The split was 80% training, 10% validation, and 10% test. The complete split statistics are shown in Table 4.1.

TABLE 4.1: Corpus Split Statistics

Split	Lines	Tokens
Train	131,067	20,247,358
Validation	16,383	2,544,782
Test	16,384	2,533,853
Total	**163,834**	**25,325,993**

Token counts are as measured by the final 2,000-token SentencePiece tokenizer. The average sequence length across all splits was approximately 154.5 tokens per line, with 87.9% of lines fitting within 512 tokens and 98.2% fitting within 1,024 tokens.

4.4 Tokenizer Design and Selection

4.4.1 Background

The tokenizer question deserves its own careful treatment. In the first experiment, a 32,000-token SentencePiece tokenizer was trained and used with BLOOMZ — this size was chosen based on what seemed standard for a multilingual model of that

scale. In the second experiment with Mini-GPT, an 8,000-token tokenizer was used, reduced based on the smaller model size. Neither of these choices was backed by empirical analysis of the actual corpus. By the time the primary experiment was being designed, it was clear that vocabulary size needed to be studied properly rather than guessed.

4.4.2 Training Tokenizers Across 25 Vocabulary Sizes

A dedicated SentencePiece BPE tokenizer was trained for each of 25 candidate vocabulary sizes: 900, 1000, 1300, 1500, 1800, 2000, 2300, 2500, 2800, 3000, 3500, 4000, 4500, 5000, 6000, 7000, 8000, 9000, 10000, 12000, 15000, 18000, 20000, 25000, and 30000 tokens. All tokenizers were trained on the 163,834-line Meitei Mayek corpus with identical configuration except for vocabulary size.

4.4.3 Coverage and Compression Analysis

Each tokenizer was run on the entire corpus with a coverage-measuring script. For every vocabulary size, the script encoded every line in the corpus and counted how many times each token ID was used. There were four metrics recorded for every vocabulary size: what percentage of tokens were seen at least once, at least 10 times, at least 100 times, and at least 1,000 times in the corpus.

1,000 occurrences was chosen as the cutoff for selecting a vocabulary size. If a token is used fewer than 1,000 times across the entire training corpus, it will not appear frequently enough for the model to learn how to use it. Any vocab where significant numbers of tokens fall below that threshold is too large for the data it will be trained on.

Coverage isn't the only factor; two measurements of compression were recorded as well: the token-to-character ratio and the average number of tokens per line. The lower the token-to-character ratio, the more information content the tokenizer packs into each token — the more language the model sees per training step. Both are important: high compression with poor coverage means the model sees overly-long

sequences, while good coverage with high token counts means many of those tokens won't actually matter because they won't be seen often enough.

One metric stood out in every one of the 25 vocabulary sizes tested: not Einzel token was marked as unknown, showing that the Meitei Mayek script is small enough that even extremely small vocabularies can cover it.

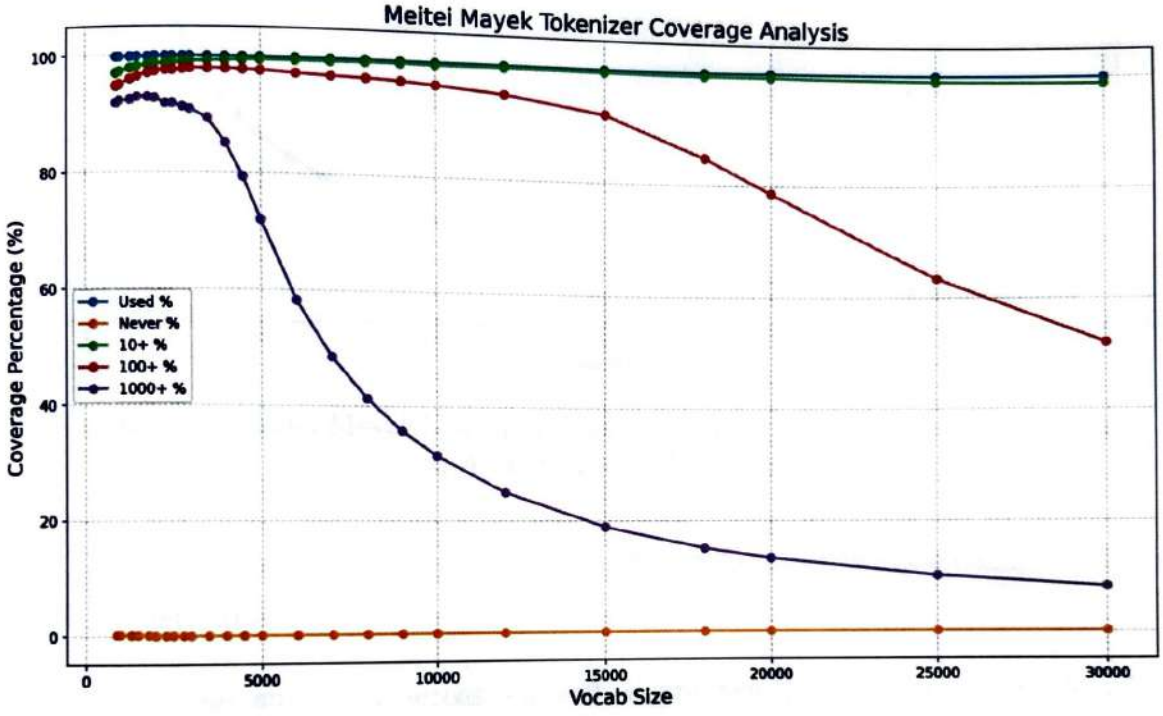


FIGURE 4.1: Meitei Mayek Tokenizer Coverage Analysis across 25 Vocabulary Sizes

The coverage plot (Figure 4.1) shows the behaviour of all four frequency thresholds across the 25 vocabulary sizes. The key line is the 1,000-occurrence coverage shown in purple. At vocabulary sizes up to around 3,000, more than 90% of tokens appeared at least 1,000 times — nearly the entire vocabulary was statistically well-represented in the corpus. Beyond 3,000 the line drops steadily: 72% at 5,000, 58% at 6,000, below 50% at 7,000, and just 8% at 30,000. This confirms that vocabulary sizes above 5,000 are clearly unsuitable for a corpus of this size.

The metrics plot (Figure 4.2) shows the compression side of the picture. The token-to-character ratio drops from 0.387 at 900 tokens to 0.312 at 2,000 tokens — a meaningful improvement. Beyond 2,000 the curve flattens noticeably. Going all the way from 2,000 to 30,000 only brings the ratio down further to 0.201 — a modest

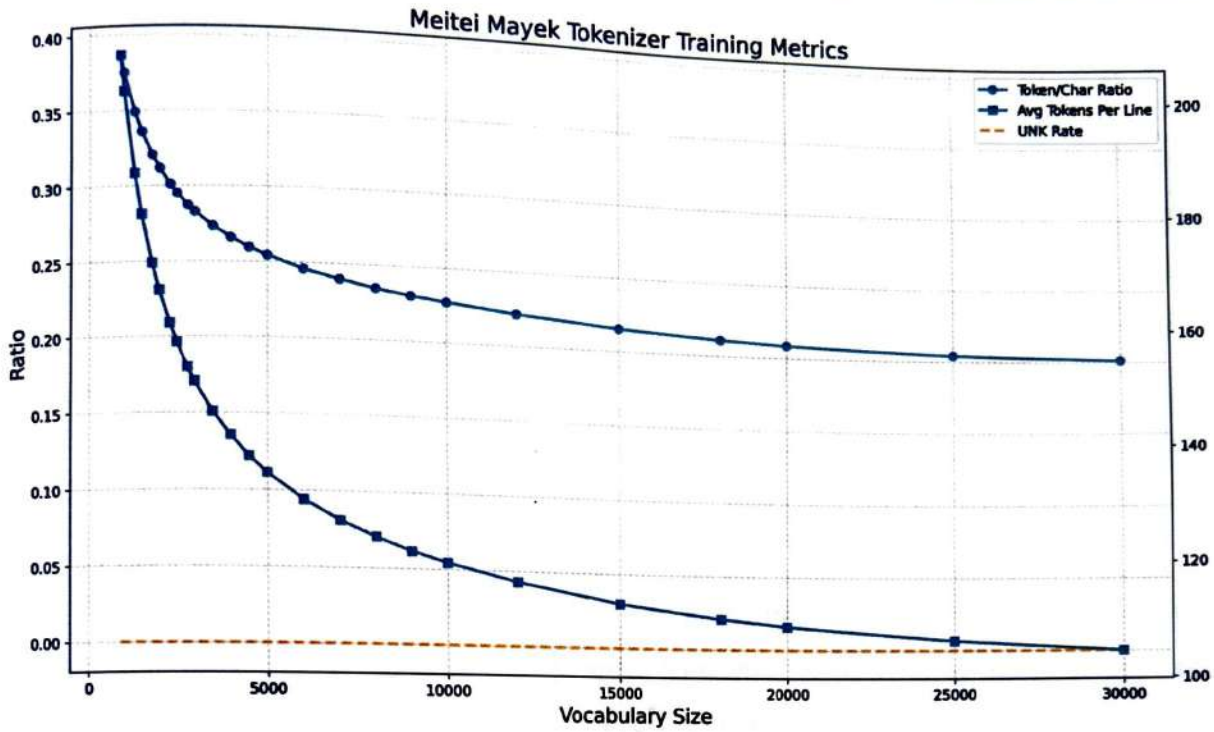


FIGURE 4.2: **Meitei Mayek Tokenizer Training Metrics — Token/Character Ratio** and Average Tokens Per Line

additional gain achieved at the cost of the 1,000-occurrence coverage collapsing from 92.7% to just 8.0%.

The full coverage summary across selected vocabulary sizes is shown in Table 4.2.

TABLE 4.2: Tokenizer Coverage Summary (Selected Vocabulary Sizes)

Vocab	Used%	10+%	100+%	1000+%
900	99.7	96.9	94.7	91.8
1000	99.7	97.2	95.0	92.2
1500	99.8	98.1	96.4	92.9
1800	99.8	98.4	97.0	92.9
2000	**99.9**	**98.6**	**97.3**	**92.7**
3000	99.9	99.0	97.8	90.8
5000	99.9	99.4	97.6	72.0
8000	99.9	99.5	96.7	41.3
15000	99.9	99.4	91.9	18.9
30000	99.9	98.7	51.9	8.0

4.4.4 Vocabulary Size Selection

A vocabulary of 2,000 tokens was selected based on both the coverage and compression evidence together. Neither metric alone would have been sufficient to make this decision confidently — a vocabulary that compresses well but has poor coverage is useless in practice, and one with excellent coverage but no compression efficiency produces sequences the model struggles to learn from efficiently.

4.4.5 Vocabulary Integration Strategy

A further decision specific to the Mistral 7B experiment was how to integrate the 2,000-token Meitei Mayek vocabulary with the base model. Two approaches were possible: replacing Mistral's original 32,768-token vocabulary entirely, or extending it by appending the 2,000 new Meitei Mayek tokens to produce a combined vocabulary. This question was investigated through a controlled experiment and is documented in full in Section 5.4.2. The result of that experiment that full replacement significantly outperformed extension, informed all subsequent training decisions.

4.5 Summary

This chapter documented the complete data pipeline from raw source collection to final tokenizer selection. Three sources provided the raw text across the project's experiments: the Hugging Face english-TO-meitei-mayek dataset yielding 401,757 cleaned sentences for the early experiments, and two newly collected sources — e-pao.net news articles and the Mendeley printed document dataset — producing a merged corpus of 163,834 lines and approximately 25.3 million tokens for the primary experiment. The e-pao.net data required a complete legacy font conversion pipeline to recover proper Unicode Meitei Mayek from ratha-encoded ASCII text.

The tokenizer for the primary experiment was selected through systematic coverage and compression analysis across 25 vocabulary sizes, with 2,000 tokens identified as the optimal balance.

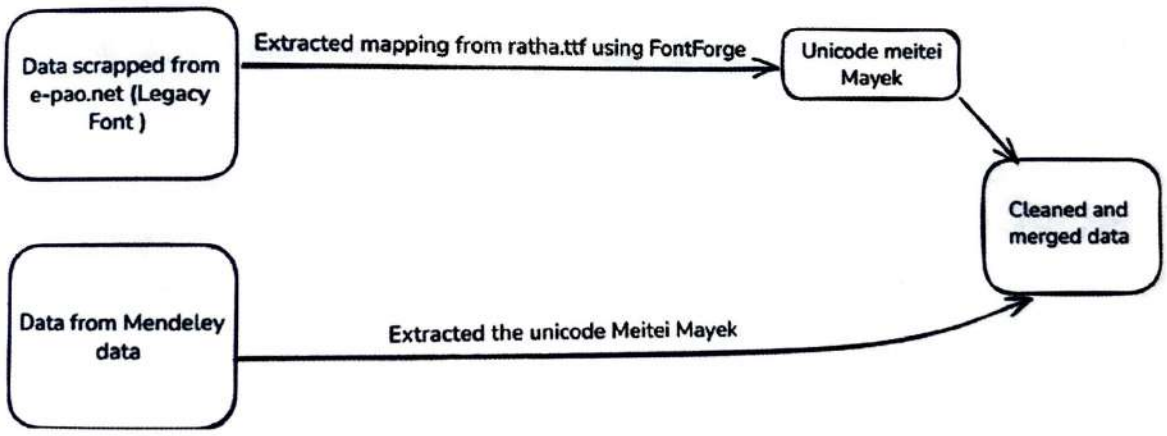


FIGURE 4.3: Data Collection Pipeline

Chapter 5

Experiments, Analysis and Results

5.1 Experimental Setup

Three different experiments were run total throughout this project. Each experiment varied what was learned from those before it. They had different model, dataset, hardware, and training config. The one thing constant about all three of them was about the goal to create a model that can generate fluent text using the Meitei Mayek script, and quantify how successful it is.

The experiments are presented in the order they were conducted. Each section describes the specific setup for that experiment, the results obtained, and the observations that motivated the design of the next experiment.

5.2 Experiment 1 — BLOOMZ-1B1 Fine-Tuning

The dataset used here was the cleaned HuggingFace english-TO-meitei-mayek corpus of 401,757 sentences described in Chapter 4. For this pilot, only a subset of 1,000 sentences was used for training. The base model was BLOOMZ-1B1 [11], a 1.1 billion parameter decoder-only Transformer from the BigScience Workshop, chosen for its multilingual pretraining and its availability in a size small enough to run on

Google Colab's free GPU tier. Training ran entirely on Colab without any dedicated hardware.

A custom SentencePiece tokenizer with a vocabulary of 32,000 tokens was trained on the full 401,757-sentence corpus and used for this experiment. The token embedding layer of BLOOMZ was resized to match this vocabulary size. LoRA was applied to the four attention projection layers — query, key, value, and output projections — with rank 8 and alpha 32. Gradient checkpointing was enabled to reduce memory usage during backpropagation. Training ran for approximately 24 epochs on the 1,000-sentence subset using the PyTorch framework with the AdamW optimizer in FP16 precision.

The training configuration is summarized in Table 5.1.

TABLE 5.1: Experiment 1 Training Configuration

Setting	Value
Base model	BLOOMZ-1B1
Parameters	1.1 billion
Dataset	1,000 sentences (pilot subset)
Tokenizer	SentencePiece, 32,000 vocab
LoRA rank / alpha	8 / 32
LoRA target modules	q, k, v, o projections
Epochs	~24
Framework	PyTorch, FP16
Hardware	Google Colab free GPU

5.2.1 Results

Insert generated Meitei Mayek output figure here

Insert generated Meitei Mayek output figure here

The training loss decreased from 7.19 to 0.67 across the training run, showing the model was actively fitting the 1,000-sentence training subset. Evaluation on the validation set produced a validation loss of 1.1649 and a perplexity of 3.21.

```
prompt = "ꯑꯁꯥꯡ ꯑꯣꯏꯕꯥ ꯑꯁꯤ"

inputs = tokenizer(prompt, return_tensors="pt")
#.to("cuda")

with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=40,
            do_sample=True,
            temperature=0.8,
            top_p=0.85,
            top_k=40,
            repetition_penalty=1.5,
            no_repeat_ngram_size=3,
            length_penalty=1.1,
            renormalize_logits=True,
            eos_token_id=tokenizer.eos_token_id,
            pad_token_id=tokenizer.pad_token_id
        )

print("\nGenerated Output:\n")
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

FIGURE 5.1: Prompting BLOOMZ-1B1 Fine-Tuning

```
Generated output : ꯑꯁꯥꯡ ꯑꯣꯏꯕꯥ ꯑꯁꯤ . ꯑꯃꯁꯨꯡꯅꯥ ꯀꯅꯥꯒꯨꯝꯕꯥ ꯑꯣꯏꯔꯕꯗꯤ ꯑꯁꯥꯑꯣꯕꯥ ꯑꯃ
ꯑꯃꯨꯛꯍꯟꯗꯨꯅꯥ - ꯑꯁꯤꯒꯤꯃꯆꯥꯁꯤꯡ ꯑꯁꯦꯡꯂꯕꯥ ꯃꯤꯑꯣꯏꯁꯤꯡ ꯃꯇꯧ ꯀꯥꯟꯅꯕꯥ ꯃꯤꯑꯣꯏꯁꯤꯡ ꯑꯃꯁꯨꯡ ꯃꯤꯑꯣꯏꯁꯤꯡꯒꯤ ꯑꯣꯏꯅꯥ ꯑꯁꯤꯒꯤ ꯃꯔꯨꯑꯣꯏꯕꯥ ꯑꯃꯁꯨꯡ
ꯀꯥꯟꯅꯤꯌꯦꯡ ꯱꯹ ꯑꯁꯦꯡꯂꯕꯥ ꯑꯃꯁꯨꯡ ꯑꯁꯥꯑꯣꯕꯥ ꯑꯣꯏꯅꯥ ꯑꯁꯤꯒꯤ ꯑꯁꯥꯑꯣꯕꯥ ꯑꯃꯁꯨꯡ ꯑꯁꯥꯑꯣꯕꯥ ꯑꯣꯏꯅꯥ ꯑꯁꯤꯒꯤ ꯑꯁꯥꯑꯣꯕꯥ ꯑꯃ
ꯑꯁꯥꯑꯣꯕꯥ ꯑꯣꯏꯅꯥ ꯑꯁꯤꯒꯤ ꯑꯁꯥꯑꯣꯕꯥ ꯑꯃꯁꯨꯡ ꯑꯁꯥꯑꯣꯕꯥ ꯑꯣꯏꯅꯥ ꯑꯁꯤꯒꯤ ꯑꯁꯥꯑꯣꯕꯥ ꯑꯃ
```

FIGURE 5.2: Sample Generated Text from BLOOMZ-1B1 Fine-Tuning

The model has produced text using Meitei Mayek characters successfully. And we can confirm that the fine-tuning pipeline was functional and that the model had learnt basic script-level patterns from the small training set.

5.2.2 Observations and Limitations

However, even with these promising statistics, there were two issues evident from the generated samples. First one is the semantic consistency was low. The model would generate fake Meitei Mayek strings which looked realistic but in actual it did not make much sense. Second, and more crucially, artifacts from English sometimes showed up in the generations, indicating that perhaps multilingual pretraining was hindering the model's capability to specialize towards a single target script.

The perplexity of 3.21 also needs to be interpreted carefully. This value was computed on a validation set drawn from the same 401,757 sentence corpus and the model was only trained on 1,000 sentences from that pool.

5.3 Experiment 2 — Custom Mini-GPT from Scratch

The second experiment took a completely different direction. Rather than fine-tuning a pretrained model, a small decoder-only Transformer was designed and trained entirely from scratch using only Meitei Mayek text.

The full 401,757-sentence HuggingFace corpus was used, split 90% for training and 10% for validation. A new SentencePiece tokenizer was trained with a vocabulary of 8,000 tokens, producing approximately 7.78 million tokens in total after tokenization.

The model architecture was a custom decoder-only Transformer implemented in PyTorch, similar to the Mini-GPT design. The configuration is shown in Table 5.2.

TABLE 5.2: Mini-GPT Architecture Configuration

Parameter	Value
Vocabulary size	8,000
Context length	64 tokens
Embedding dimension	512
Attention heads	8
Transformer layers	6
Dropout	0.1
Total parameters	~27.1 million

Training used the AdamW optimizer with a learning rate of 3×10^{-4}, a batch size of 64, and gradient clipping at 1.0. The model was trained for 3 epochs on an NVIDIA RTX 4060 Laptop GPU with 8GB of VRAM. Total training time was approximately 12 hours and 25 minutes across the three epochs — Epoch 1 took around 3 hours 20 minutes, Epoch 2 approximately 5 hours 13 minutes, and Epoch 3 around 3 hours 52 minutes.



FIGURE 5.3: Training Progress of the Mini-GPT Model

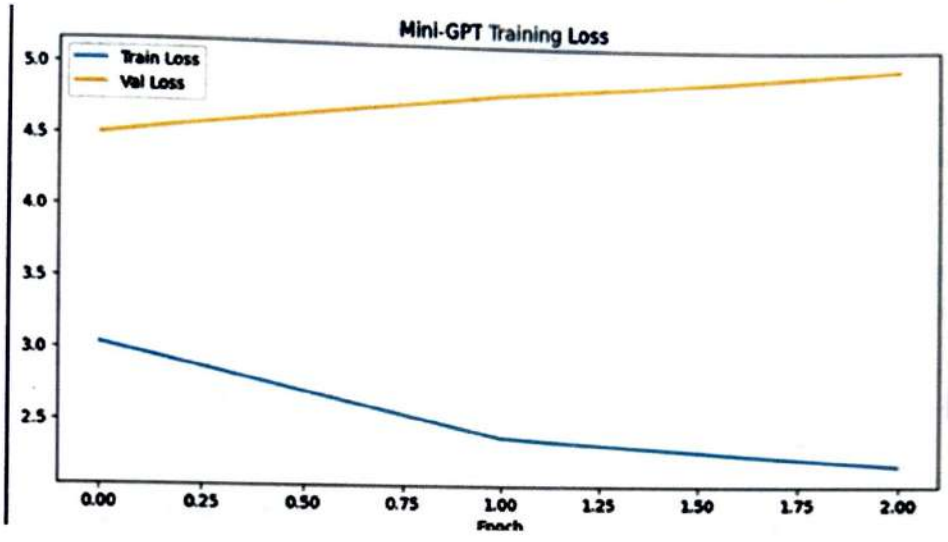


FIGURE 5.4: Training and Validation Loss Curves for Mini-GPT

5.3.1 Results

The training loss decreased consistently across all three epochs, showing that the model was successfully learning patterns from the Meitei Mayek training data. However, the validation loss increased across epochs — a clear sign of overfitting. The model was memorizing the training distribution rather than learning generalizable language patterns.

Generation Configuration :

- **Context Length: 64 tokens**
- **Max New Tokens: 100**
- **Temperature: 0.7**
- **Top-k Sampling: k = 50**

Prompt: ꯑꯁꯤꯒꯤ
Generated: ꯑꯁꯤꯒꯤ ꯂꯩꯍꯟꯗꯕ ꯑꯃꯁꯨꯡ ꯑꯆꯨꯝꯕ ꯁꯦꯡꯂꯕꯥ ꯑꯃꯅꯥ ꯄꯨꯟꯁꯤꯅꯥ ꯑꯃꯁꯨꯡ ꯑꯃꯁꯨꯡ ꯑꯃꯁꯨꯡ

two architectural improvement. One is Sliding Window Attention, which allows the model to handle longer sequences more efficiently by limiting direct attention to a local window and still propagating information through layer stacking simultaneously. And another one is Grouped Query Attention, which reduces the memory required for the key-value cache during generation by sharing key and value projection heads across groups of query heads.

Prior work [13] confirms that Mistral 7B can be used effectively to local language corpora through continued pretraining. This provides evidence that the architecture is well-suited for low-resource language specialization. Taking all of these factors together i.e benchmark efficiency, architectural design, memory suitability, and demonstrated adaptability, we chose Mistral 7B as the base model for the primary experiment.

5.4.2 Tokenizer Strategy: Replacement vs Extension

Before committing to the full Mistral 7B training run, a specific design question needed to be settled empirically: should Mistral's original 32,768-token vocabulary be completely replaced with the 2,000-token Meitei Mayek vocabulary, or should the Meitei Mayek tokens be appended to the existing vocabulary to produce a combined 32K+2K vocabulary?

Two separate training scripts were written — one implementing full replacement and one implementing extension. The extension approach initialized new token embeddings with the mean of Mistral's existing embedding weights, which is standard practice for adding tokens to a pretrained model. Both scripts used identical configurations: LoRA with rank 64 and alpha 128, applied to the same seven module types, with the same learning rate of 2×10^{-4}, batch size of 4, gradient accumulation of 8, and a fixed limit of 1,000 training steps.The same train and validation splits from the 163,834-line corpus were used for both.

TABLE 5.3: Tokenizer Strategy Comparison Results at 1,000 Steps

Approach	Final Train Loss	Final Val Loss	Final Perplexity
2K Replacement	0.636	5.03	153
32K+2K Extension	0.741	5.88	356

5.4.2.1 Results

The replacement approach outperformed extension on every metric (Table 5.3). Validation perplexity was 153 against 356. The initial gradient norm spike was also notably different: the replacement model peaked at around 24 while the extension model spiked to approximately 56 in the first 200 steps. The much larger spike in the extension model reflects the difficulty of adapting 32,000 pretrained embeddings that have no connection to Meitei Mayek while simultaneously learning 2,000 new ones.

It should be noted that the perplexity values in both experiments are likely slightly lower than the true values, because sequences shorter than 512 tokens were padded with zeros and the model was scored on predicting those padding positions as well. Since padding is always the same token, the model learns it trivially and gets easy credit on those positions. However, since both scripts used identical padding behaviour, this affects both results equally and does not change the conclusion.

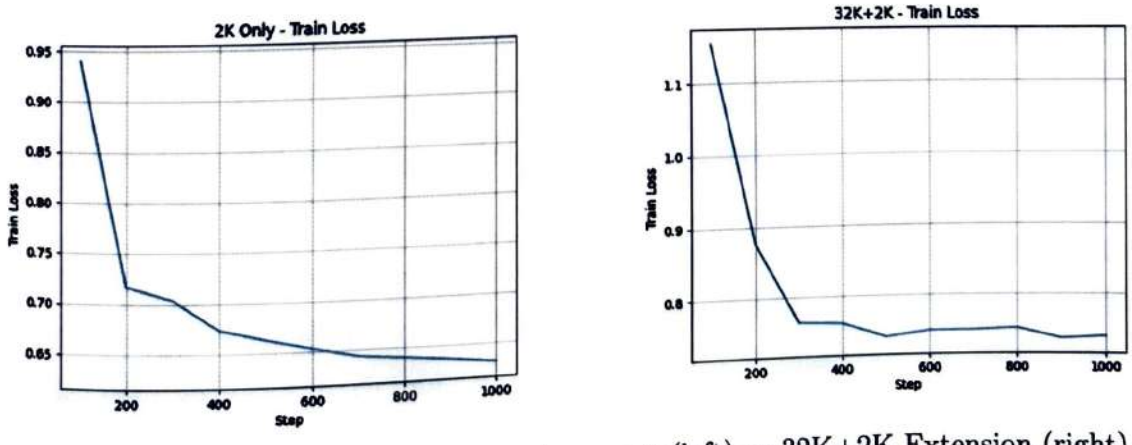


FIGURE 5.6: Training Loss: 2K Replacement (left) vs 32K+2K Extension (right)

Full vocabulary replacement was adopted for all subsequent training. The 32,000 original Mistral tokens are irrelevant when the entire training corpus is in Meitei

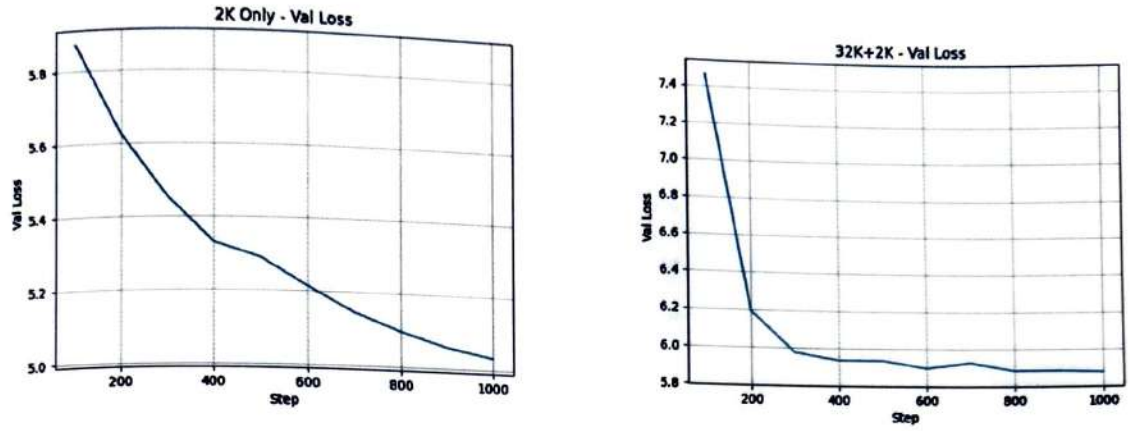


FIGURE 5.7: Validation Loss: 2K Replacement (left) vs 32K+2K Extension (right)

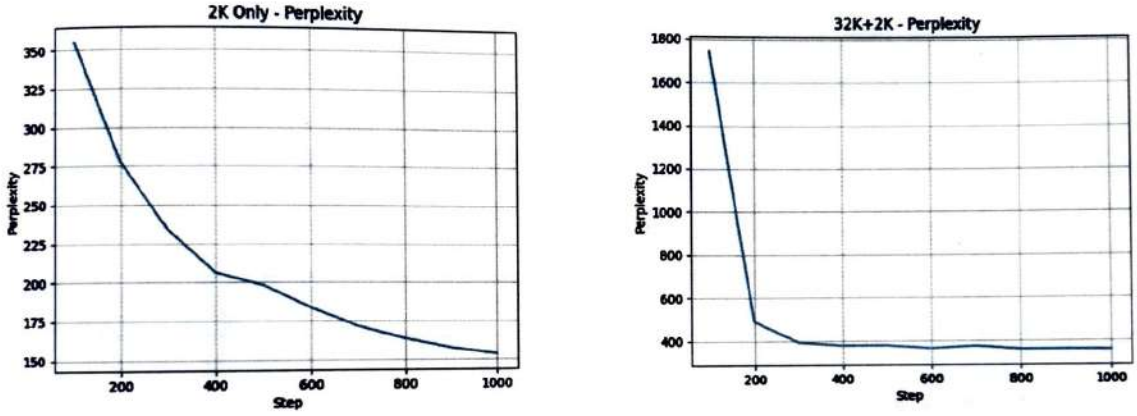


FIGURE 5.8: Validation Loss: 2K Replacement (left) vs 32K+2K Extension (right)

Mayek — they add computational overhead and appear to actively impede the model's ability to adapt.

5.4.3 Mistral 7B Main Training Setup

The primary experiment fine-tuned Mistral 7B on the full 163,834-line Meitei Mayek corpus using the 2,000-token vocabulary replacement strategy confirmed in the tokenizer comparison experiment. The corpus was split into training, validation, and test sets at an 80/10/10 ratio, producing 131,067 training lines, 16,383 validation lines, and 16,384 test lines. The total token count across all splits was approximately 25.3 million tokens — 20.2 million in the training set, 2.5 million in validation, and 2.5 million in the test set, as measured by the 2,000-token SentencePiece tokenizer. Training was conducted on an NVIDIA RTX PRO 5000 Blackwell GPU with 48GB

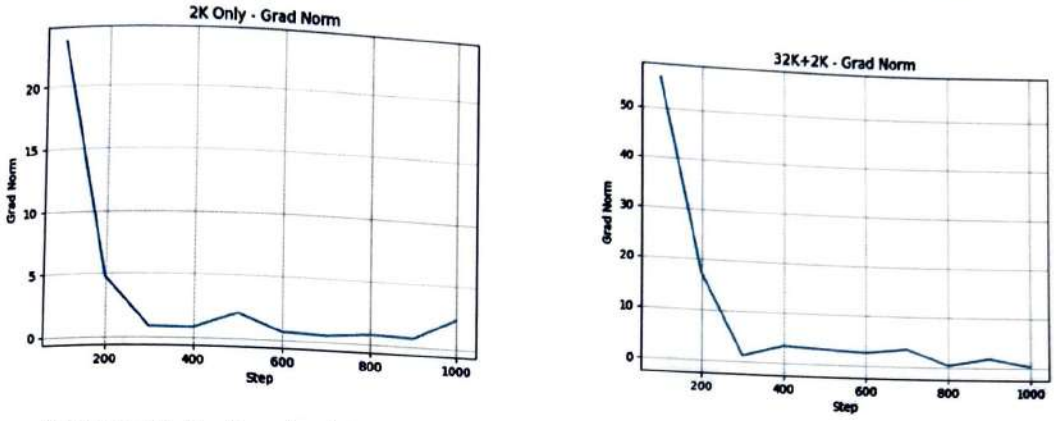


FIGURE 5.9: Perplexity: 2K Replacement (left) vs 32K+2K Extension (right)

of VRAM, provided by the project supervisor. The GPU allocated approximately 14.4 GB throughout training, well within the available memory.

The complete training configuration is summarized in Table 5.4.

TABLE 5.4: Mistral 7B Training Configuration

Setting	Value
Base Model	Mistral 7B
Tokenizer	2K SentencePiece BPE (full replacement)
LoRA rank / alpha	32 / 64
LoRA dropout	0.05
Target modules	q, k, v, o, gate, up, down projections
Trainable parameters	83,886,080 (1.18% of 7B)
Effective batch size	32 (batch 2 × grad accum 16)
Max sequence length	1,024 tokens
Peak learning rate	2×10^{-5} with cosine decay
Warmup steps	400
Gradient clipping	0.5
Total steps	10,000
GPU memory	~14.4 GB / 48 GB available

5.4.4 Training Results

The training metrics across selected steps are shown in Table 5.5.

Train loss dropped from 11.94 at step 100 to 5.58 at step 10,000, a smooth and consistent descent throughout. The steep initial drop in the first 500 steps reflects

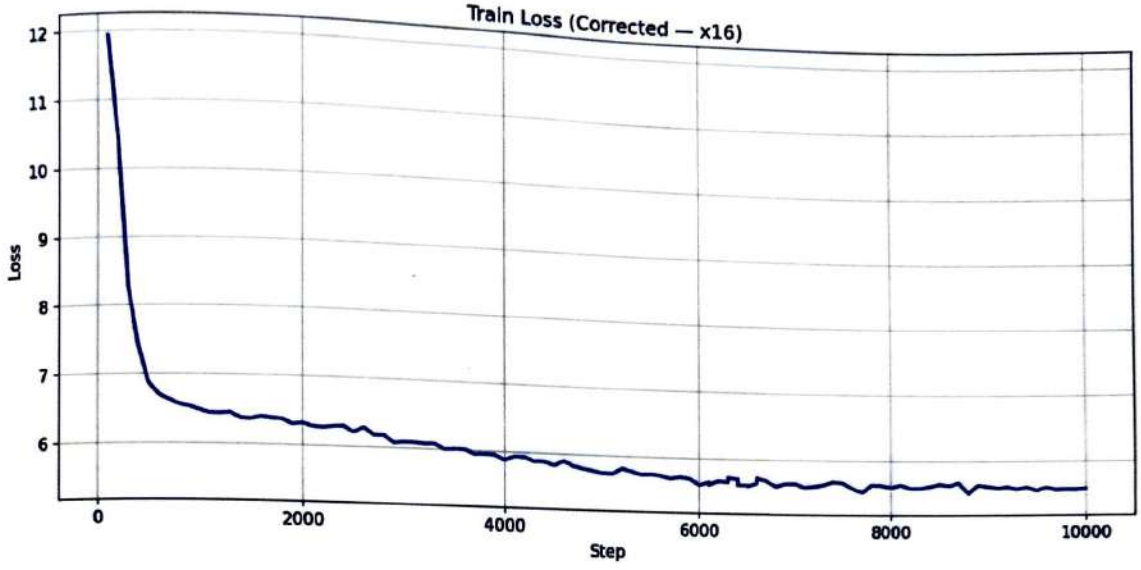


FIGURE 5.10: Mistral 7B Training Loss over 10,000 Steps

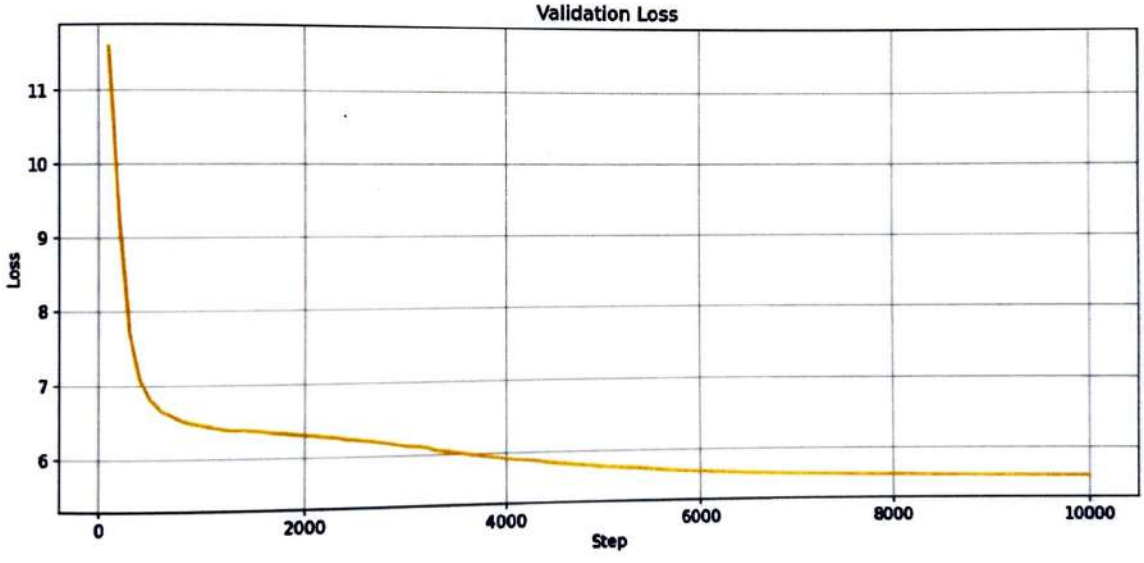


FIGURE 5.11: Mistral 7B Validation Loss over 10,000 Steps

the model rapidly adjusting its randomly initialized embedding layer to the Meitei Mayek token space. The descent then slowed and became more gradual from around step 1,000 onward, as the model moved from coarse adjustment to fine-grained optimization.

Validation loss tracked the training loss almost exactly throughout all 10,000 steps. The gap at the final step was only 0.03 ,train loss 5.58, validation loss 5.61. This near-identical behaviour of the two curves is one of the strongest results of the

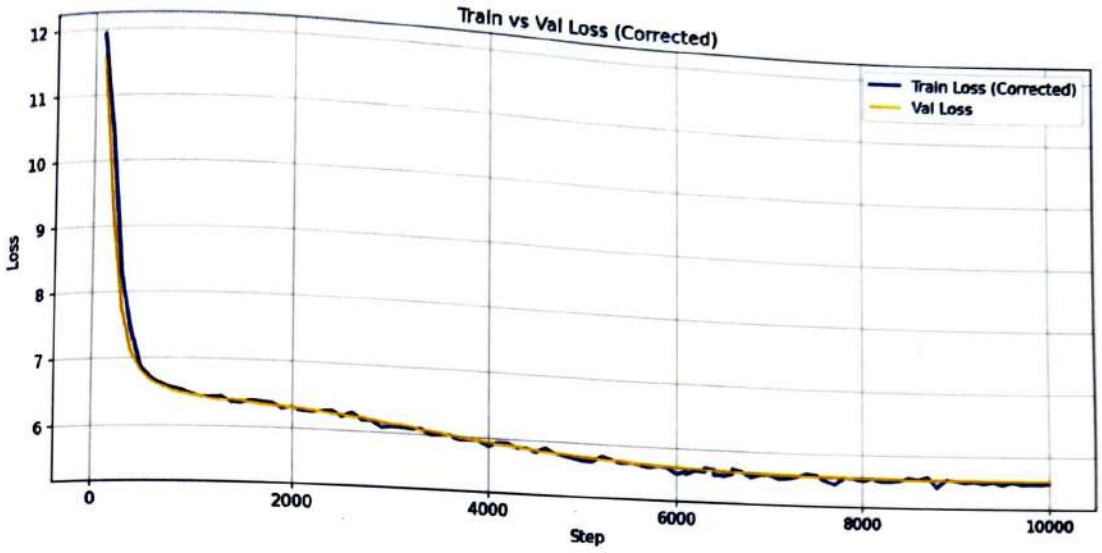


FIGURE 5.12: Train vs Validation Loss — Near-zero gap confirms no overfitting

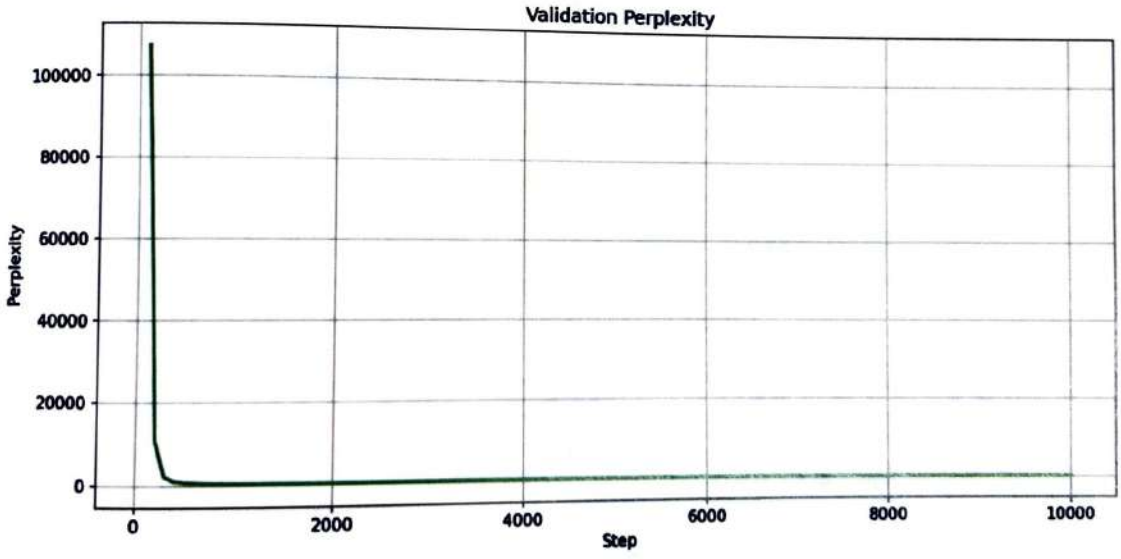


FIGURE 5.13: Validation Perplexity over 10,000 Steps

experiment ,the model was not memorizing the training data but genuinely learning patterns that transferred to unseen text.

Perplexity fell from 107,397 at step 100 to 273 at step 10,000. The curve was still slowly descending at the end of training, indicating the model had not fully converged and would likely continue to improve with additional steps and data.

Gradient norm showed an initial spike of approximately 43 at around step 200. This

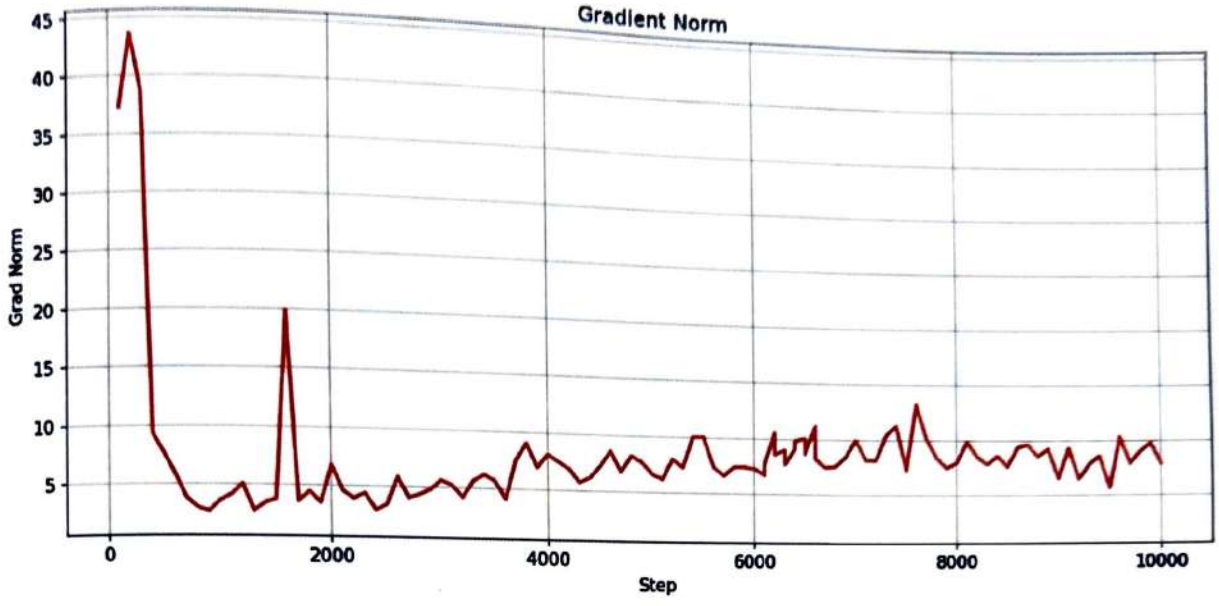


FIGURE 5.14: Gradient Norm over 10,000 Steps

TABLE 5.5: Mistral 7B Training Metrics at Selected Steps

Step	Train Loss	Val Loss	Perplexity	Grad Norm
100	11.94	11.58	107,397	37.28
500	6.88	6.82	919	7.64
1,000	6.48	6.47	643	3.65
2,000	6.32	6.31	549	6.74
3,000	6.09	6.13	459	5.62
5,000	5.74	5.79	326	6.66
7,000	5.64	5.65	284	9.96
9,000	5.61	5.62	275	6.53
10,000	**5.58**	**5.61**	**273**	**7.86**

is expected, the randomly initialized embedding layer produces large gradients early in training as it adapts from random values. A secondary smaller spike of around 20 appeared at approximately step 1,600. From step 2,000 onward the gradient norm settled into a stable range of approximately 5 to 13 for the remainder of training, indicating healthy and stable optimization throughout.

5.5 Final Evaluation and Baseline Comparison

5.5.1 Setup

After the training was complete, an evaluation was conducted on the held-out test set. We evaluated the base model 7B with 2k vocab tokenizer and trained model 7B with 2k vocab tokenizer. We evaluated both the models on 500 lines of the held-out test set.

5.5.2 Results

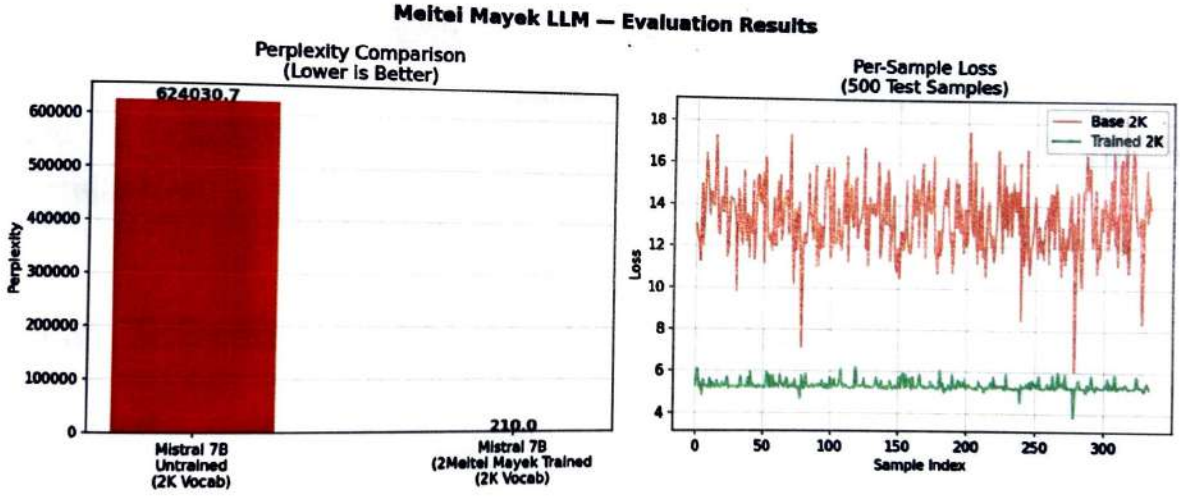


FIGURE 5.15: Perplexity Comparison (left) and Per-Sample Loss (right)

TABLE 5.6: Final Evaluation: Untrained Baseline vs Trained Model

Model	**Loss**	**Perplexity**
Mistral 7B Untrained (2K vocab)	13.34	624,031
Mistral 7B Trained (2K vocab)	5.35	210
Improvement	—	**99.97%**

The untrained baseline came out with a perplexity of 624,031. Even a completely random model that assigned equal probability to every token in a 2,000-token vocabulary would only reach a perplexity of 2,000 so the untrained model is doing far worse than random guessing. The reason for this is straightforward: when we

replaced Mistral's original 32,768-token vocabulary with our 2,000 Meitei Mayek tokens, the model's internal weights were still calibrated for the old token IDs. It was essentially being asked to read a language it had never seen, using a codebook it no longer had. The result was near-total confusion, reflected in the extremely high loss values.

The trained model scored a perplexity of 210 on the same 500 unseen test sentences. So there is a reduction of 99.97% from the baseline. This result was achieved on text the model had never seen during training. So it confirms that the learning generalized beyond the training set.

The per-sample loss chart is very useful. The untrained model's red line fluctuates wildly between approximately 10 and 18 across all 500 samples. The trained model's green line sits steadily between approximately 5.0 and 5.8 across all 500 samples with remarkably little variance. This steadiness indicates that the model performs consistently across different text samples which indicates sign of genuine generalization.

5.5.3 Inference and Generated Samples

We wrote a Python script to interact with the trained model with decoding modes: temperature-based sampling with configurable top-p sampling. This allowed real-time adjustment of generation parameters including temperature, top-p, repetition penalty, and no-repeat n-gram size, and also logged all prompts and outputs to a log file.

A separate Pyhton script was also run on the untrained base Mistral 7B with its original 32,768-token vocabulary. This provide a qualitative baseline for comparison. The contrast between the two is huge and immediately visible.

The untrained base model, when given a Meitei Mayek prompt, responded entirely in English, Korean— producing blog post headers, academic conference announcements, and code snippets. It had no understanding of the Meitei Mayek script and treated the input characters as meaningless tokens to be ignored, defaulting to English and other output from its pretraining distribution.

The trained model responded exclusively in Meitei Mayek script with no English contamination across all tested prompts and decoding modes. Selected outputs from the inference log are shown below :

```
PROMPT   : ꯏꯕꯍꯧ
SETTINGS : mode=sample | temp=0.8 | top_p=0.9 | runs=1 | repen=1.1 | ngram=3
  Run 1: : ꯃꯍꯩꯁꯤꯟꯗꯤ ꯃꯑꯣꯡꯕꯥꯏꯔꯤ ꯀꯥꯁ ꯀꯥꯇꯔꯤꯛ ꯑꯁꯤꯅꯑꯣꯏ ꯃꯥꯃꯇꯥꯡꯗꯥ
```

FIGURE 5.16: Generated text by trained model using 2k vocab size

```
PROMPT   : ꯑꯀꯤꯃꯥꯟꯅꯇꯤ ꯃꯥꯟꯅꯐ ꯏꯆꯦꯜ ꯃꯦꯡꯕꯣꯏꯗ ꯀꯤꯝꯅꯕꯥ ꯑꯁꯤꯃꯥꯟꯅꯃꯤꯠ ꯑꯀꯤꯃꯥꯟꯅꯃꯀꯤ ꯗ ꯃꯦꯡꯕ
SETTINGS : mode=sample | temp=0.7 | top_p=0.95 | runs=1 | repen=1.3 | ngram=4
  Run 1:
        if(data.length >= 15):
            print("이미지가 너무 크잖아요?")
            return False, data
# https://www.pyimagesearch.com/2016/03/14/convert-image-to-gray-scale-using-op
```

FIGURE 5.17: Generated text by untrained model using original 32k vocab size

5.6 Discussion

The results across the experiments tell a consistent and instructive story about what works for generative modelling in a low-resource indigenous script setting.

Fine-tuning a multilingual pretrained model on a tiny pilot corpus, as in Experiment 1, is enough to produce script-correct output but not enough for semantic coherence. The multilingual pretraining of BLOOMZ brings general language structure knowledge, but 1,000 sentences of Meitei Mayek fine-tuning data is far too little to redirect that knowledge meaningfully toward the target language. The English artifacts in the output reflect the dominance of English in BLOOMZ's pretraining distribution breaking through the thin layer of Meitei Mayek adaptation.

Training from scratch, as in Experiment 2, eliminates the interference from other languages entirely — the outputs were clean Meitei Mayek with no English contamination. But it introduced a data sufficiency problem that could not be resolved with the available corpus. When we trained a 27.1 million parameter model trained on 7.78 million tokens, we can see it clearly overfitting and it is justified by Chinchilla scaling laws [5].

One thing this project made clear is that choosing a tokenizer vocabulary size is not we should not make by intuition. We tested 25 different sizes ranging from 900 to 30,000 tokens and measured how well each one actually covered the corpus. Zero unknown token is achieved across all vocab sizes.The coverage and compression analysis across 25 candidate sizes showed clearly that vocabularies above 5,000 tokens are poorly matched to a corpus of this size. So we chose 2000 vocab size based on these findings.

The experiment about the tokenizer strategy has produced a very clear results of the project. Full vocabulary replacement outperformed extension by a factor of more than two in perplexity — 153 against 356 — under identical training conditions.

A large capable base model, fully replaced with 2,000-token vocabulary tokenizer, a diverse 163,834-line corpus, and parameter-efficient fine-tuning with a configured LoRA produced a model that generalized cleanly. The data about train-validation gap of only 0.03 across 10,000 steps is strong evidence of this. The 99.97% perplexity reduction from baseline on unseen test data confirms that the training was learning transferable structure of the Meitei Mayek language.

The main limitation that runs through all experiments is data. Even the primary experiment's 25 million tokens is very small by the standards of modern language model training. The perplexity curve was still descending at step 10,000, meaning the model had not converged ,more data and more training would almost certainly produce further improvements.

5.7 Summary

This chapter presented the various experiments conducted during the project. Experiment 1 established that fine-tuning BLOOMZ-1B1 on a 1,000-sentence Meitei Mayek pilot produced script-correct output but limited semantic coherence, with English artifacts appearing in generated text. Experiment 2 showed that training a 27.1 million parameter custom model from scratch on the full corpus produced cleaner Meitei Mayek output but suffered from clear overfitting due to the small dataset size relative to model capacity.

The vocabulary size selection study established 2,000 tokens as the optimal vocabulary for the available corpus through systematic coverage and compression analysis across 25 candidate sizes. A further controlled experiment confirmed through direct comparison that full vocabulary replacement outperforms vocabulary extension for adapting Mistral 7B to Meitei Mayek, with a perplexity of 153 against 356.

The primary training run fine-tuned Mistral 7B for 10,000 steps using LoRA on the 163,834-line corpus totalling approximately 25.3 million tokens, achieving a final train loss of 5.58 and validation loss of 5.61 with a gap of only 0.03. Final evaluation on 500 unseen test samples showed a perplexity reduction of 99.97% from the untrained baseline,from 624,031 to 210. Generated outputs were exclusively in Meitei Mayek script with no English artifacts across all tested prompts and decoding configurations.

Chapter 6

Conclusions and Practical Implications

6.1 Study Findings

This project set out to address a specific and well-defined gap: the complete ab sence of any generative language model for the Manipuri language in its native Meitei Mayek script. The work moved through three model experiments and several supporting studies, producing a trained model, a reusable corpus, a validated tok enizer, and a documented methodology. The findings from each stage of this work are summarized here.

The first experiment confirmed that fine-tuning a multilingual pretrained model on even a small Meitei Mayek corpus is enough to produce script-correct output. BLOOMZ-1B1, fine-tuned on just 1,000 sentences, generated Meitei Mayek charac- ters and showed it had absorbed basic script patterns. However, the outputs lacked semantic coherence and occasionally contained English text — a sign that the multi lingual pretraining was interfering with the limited Meitei Mayek adaptation signal.

The second experiment showed that training a small model from scratch on Meitei Mayek data alone produces cleaner script output with no English contamination, but suffers from overfitting when the corpus is too small relative to the model's parameter count. A 27.1 million parameter model on 7.78 million tokens was well

into the overfitting regime, with validation loss rising even as training loss continued to fall.

The vocabulary size study demonstrated that tokenizer design for a low-resource script deserves the same rigorous empirical treatment as model selection. Systematic coverage and compression analysis across 25 candidate vocabulary sizes showed thatvocabularies above 5,000 tokens are poorly suited to a corpus of this size, and that 2,000 tokens provided the best balance of coverage quality and compression efficiency with zero unknown tokens confirmed across all data splits.

The tokenizer strategy comparison showed clearly that fully replacing Mistral 7B's original vocabulary with the 2,000-token Meitei Mayek vocabulary outperforms ex tending the existing vocabulary, with a final perplexity of 153 against 356 under identical training conditions. For a completely unseen script, the pretrained em beddings of the original vocabulary provide no useful signal and actively impede adaptation.

The primary experiment demonstrated that a 7 billion parameter model can be meaningfully adapted to a completely unseen script using parameter-efficient fine tuning on a modest corpus, provided the tokenizer is purpose-built and the vocab ulary strategy is correct. Training Mistral 7B with LoRA for 10,000 steps on the 163,834-line corpus produced a model with a train-validation gap of only 0.03 — confirming generalization rather than memorization. Evaluation on 500 unseen test samples showed a perplexity reduction of 99.97% from the untrained baseline i.e from 624,031 to 210. Generated text was exclusively in Meitei Mayek script with no English contamination across all tested prompts and decoding configurations.

6.2 Research Contributions

This project delivered several concrete things for low-resource NLP and Manipuri language technology.

First, there's now a trained, fully-documented generative language model for Meitei Mayek. That gives anyone who follows a baseline to compare to—which simply didn't exist before.

Second, we built a new cleaned corpus for Manipuri in the Meitei Mayek script. This combined 17+ years of e-pao.net news articles and the Mendeley printed documents dataset, giving a rich, diverse collection that researchers didn't have before. The font conversion pipeline we wrote for legacy Ratha-encoded text on e-pao.net is also reusable. Anyone working with that old data now has a way to make it usable.

Third, the 2,000-token SentencePiece BPE tokenizer isn't just another tool—it's the first one designed and validated specifically for model training in Meitei Mayek.

The methodology itself is also a contribution. The project documented a complete end to-end pipeline — from identifying data sources through font conversion, corpus cleaning, tokenizer selection, vocabulary strategy comparison, model fine-tuning, and evaluation — in sufficient detail that it can be replicated and extended. For other low-resource languages facing similar combinations of data scarcity, legacy encoding problems, and absence of NLP infrastructure, this pipeline provides a practical template.

Finally, the vocabulary replacement versus extension experiment contributes a con crete empirical finding to the broader question of how to adapt large language models to completely unseen scripts. The result that full replacement significantly outper forms extension when the target script has no overlap with the base model's pre training vocabulary — is a finding that would be relevant to any team working on a similar adaptation task.

6.3 Practical Implications

The practical significance of this work extends beyond the academic contribution. Manipuri is the official language of the state of Manipur and one of India's 22 constitutionally scheduled languages. Meitei Mayek is its official and educationally adopted script. Yet every AI language tool that Manipuri speakers encounter — autocomplete, spell check, document summarization, search, operates in languages other than their own. This project takes a first step toward changing that.

In **education**, a functional generative model for Meitei Mayek opens the possibility of building writing assistance tools for Meitei Mayek medium schools and universi

ties, where students currently have no AI support in their language of instruction. Even at the current level of capability, a model that can generate plausible Meitei Mayek text can serve as a component in spell checking, text completion, or simple content generation tools.

In **cultural preservation**, the corpus construction methodology developed here, particularly the web scraping and font conversion pipeline, can be applied to digitize and process further Meitei Mayek text archives that currently exist only in legacy encoded formats. Making this material computationally accessible is a prerequisite for building better models and for applying NLP tools to historical and literary analysis of Manipuri texts.

And **public services** i.e state government, universities , any institute,if they want to use any digital tools in Meitei Mayek, they will eventually needs a language model as its core. Chatbots, document processors, search engines etc, they all start with this foundation.

6.4 Study Limitations and Scope for Future Work

Data is still the core bottleneck. Approximately 25 million tokens is very small for a 7 billion parameter model. The perplexity curve was still descending at step 10,000, indicating the model had not converged. More training data would almost certainly continue to improve performance. The corpus is also predominantly news text ,the model has been exposed to journalistic Meitei Mayek but has had no training on literary prose, conversational text, social media language, or formal administrative writing. A model trained only on news text will generate text that sounds like news, which limits its general usefulness.

The evaluation is purely quantitative. Perplexity measures how well the model predicts the next token but it does not measure whether the generated text is seman tically meaningful, grammatically correct by native speaker standards, or culturally appropriate. A human evaluation by native Manipuri speakers assessing the gener ated outputs on coherence, fluency, and appropriateness was planned but could not be completed within the scope of this project. This remains an important next step before the model can be considered ready for any practical deployment.

Hardware and infrastructure are a serious constraint for future work. The primary experiment in this project was only possible because of access to a supervisor-provided NVIDIA RTX PRO 5000 Blackwell GPU with 48GB of VRAM. Without that resource, training Mistral 7B would not have been feasible at all. Tak ing this work meaningfully further i.e training on a larger corpus, running for more steps until full convergence, experimenting with larger models or full fine-tuning rather than LoRA would require access to multi-GPU setups or cloud compute clus ters that are well beyond the reach of most individual researchers or university groups in the region. This is not a limitation unique to this project. It reflects a broader structural inequality in AI research infrastructure that disadvantages insti-tutions in low-resource settings. And they are often the institutions best placed to work on low-resource languages. To address this we require institutional investment, cloud compute grants, or collaborative arrangements with better-resourced research groups.

In the future, the following can be some obvious next steps:

- Gather more text: books, government documents, social media, and chats, we need to expand the corpus.
- Train for more steps with a proper learning rate schedule to squeeze more out of the existing data.
- Try different base model and tuning methods to understand whether the lim-itations are coming more from the model or from the data.
- Conduct a real human evaluation with native Manipuri speakers to assess fluency and coherence.
- Build actual end-user applications such as writing assistant, chatbot, or text classifier to demonstrate the practical value of the work.

This project is just the start. Now the community may take what's been built and see where it can go next in the future. The foundation stone has been laid.

References

[1] BigScience Workshop. Bloomz & mt0: Pushing cross-lingual generalization to the limit. arXiv preprint arXiv:2211.01786, 2022.

[2] Steven Bird. Decolonising speech and language technology. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 3504–3519, 2020.

[3] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, 2020.

[4] E-Pao. E-pao: Manipur's online media. Online, 2026.

[5] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[6] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[7] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[8] Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6282–6293, 2020.

[9] Mendeley Data. A benchmark dataset for printed meitei/meetei script character recognition. Mendeley Data, 2021.

[10] Thoudam Doren Singh and Sivaji Bandyopadhyay. Statistical machine translation of english-manipuri using morpho-syntactic and semantic information. In *Conference of the Association for Machine Translation in the Americas*, 2010.

[11] Dayananda Thokchom. english-to-meitei-mayek. Hugging Face Datasets, 2023.

[12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[13] Husein Zolkepli, Aisyah Razak, Kamarul Adha, and Ariff Nazhan. Large Malaysian language model based on Mistral for enhanced local language understanding. *arXiv preprint arXiv:2401.13565*, 2024.

Submission ID trn:oid:::27787:139248707

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

› Bibliography

Match Groups

- 93 Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- 3 Missing Quotations 0%
  Matches that are still very similar to source material
- 1 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 1 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
4% Publications
8% Submitted works (Student Papers)

Submission ID trn:oid:::27787:139248707

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.